VIETNAM - KOREA UNIVERSITY OF INFORMATION AND COMMUNICATION TECHNOLOGY (VKU)

MINI-PROJECT SHORT TECHNICAL REPORT

Course: Cross-Platform Mobile App Development (VKU)

Mini-Project Title: Mini-Project 3 — On-Device OCR Receipt Expense Tracker

Team / Student Name: Nguyen Hoang Long (ID: 23IT.B119) | **Submission Date:** 19/10/2026

1. GENERAL INFORMATION & DELIVERABLE LINKS

- **Team Members:** Nguyen Hoang Long — Student ID: 23IT.B119 — Role: Solo Developer (Flutter Frontend, ML Kit OCR, Regex Heuristics, Custom Canvas, SQLite Backend) — Contribution: 100%
- 📦 **Signed Release APK:** `release/app-release.apk` (40.0 MB, RSA-2048 Signed & Verified via `apksigner`)
- 💻 **GitHub Repository:** https://github.com/TieuLong07/receipt-expense-tracker
- 📄 **Technical Report PDF:** `release/VKU_MiniProject3_Report.pdf` (Official Technical PDF)
- 🎥 **Video Demo (Optional):** — (link submitted separately if applicable)

2. FEATURE IMPLEMENTATION CHECKLIST

#	Required Feature	Status	Implementation Details & Acceptance Level
1	**Camera & Gallery Capture**	✅ Complete	Integrated `image_picker: ^1.2.0`. Supports capturing receipts directly via camera or choosing from gallery with 90% JPEG compression. Handled camera runtime permissions.
2	**On-Device ML Kit OCR**	✅ Complete	Uses `google_mlkit_text_recognition: ^0.15.0` Latin model. Runs 100% locally on device CPU in ~350ms with zero network dependency and strict user privacy.
3	**Heuristic Regex Parser**	✅ Complete	Custom `ReceiptRegexParser` extracts Total, Date, Merchant, and Category. Features tender exclusion (filters out `tiền mặt`, `cash`), multi-currency format support (`68.000đ`, `125,000 VND`), and fallback to largest monetary value. Unit tests pass 100%.
4	**Custom Canvas: Donut Chart**	✅ Complete	100% `CustomPainter` (zero chart libraries). Computes sweep angles from category totals, 0.04 rad gap between slices, 900ms `Curves.easeOutCubic` sweep animation, central total spent label, and responsive category pill legends.
5	**Custom Canvas: Weekly Bar Chart**	✅ Complete	100% `CustomPainter`. Renders 7-day expense trend with rounded bar tops (`RRect`), 15% headroom scaling, 3 horizontal dotted grid guides, highlight accent for current day, and compact value labels (`45k`).
6	**State Management (Provider)**	✅ Complete	`ChangeNotifierProvider` (`ExpenseProvider`) managing reactive state: receipts list, loading indicators, theme mode, category totals, weekly data calculations, and sync operations.
7	**Persistent Local SQLite DB**	✅ Complete	`sqflite: ^2.4.1` with `receipts` table. Complete CRUD: insert, query ordered by date, update after manual correction, delete with file cleanup. Serves as single source of truth (SSOT).
8	**Local Photo Storage & SSOT**	✅ Complete	Original receipt images persisted to `getApplicationDocumentsDirectory()/receipt_photos/` with UUID-based unique filenames. Zero network required for saving and browsing receipts.

#	Required Feature	Status	Implementation Details & Acceptance Level
9	**Offline-First Sync Queue**	✅ **Complete**	Receipts saved while offline marked with `sync_status = 'pending_sync'`. AppBar sync button with badge count triggers cloud sync simulation and marks records as `synced`.
10	**UI/UX Polish & Review Dialog**	✅ **Complete**	Material 3 theme with deep Teal VKU seed palette (`#0F766E`), instant Dark/Light mode toggle, no distracting emoji in UI, and a mandatory `ReviewReceiptDialog` for manual inspection/correction.
11	**Signed Release APK Packaging**	✅ **Complete**	Signed release APK (`app-release.apk`) generated with RSA-2048 SHA-256 certificate, verified by `apksigner` (v1 & v2 schemes valid), accompanied by clean git history and technical PDF.

3. TECHNICAL ARCHITECTURE & PROJECT STRUCTURE

```
receipt_expense_tracker/
├── android/
│   ├── app/
│   │   ├── build.gradle                # minSdk 21, targetSdk 34, release signingConfig, ProGuard rules
│   │   ├── proguard-rules.pro          # -dontwarn com.google.mlkit.** & keep rules
│   │   ├── release.keystore            # RSA 2048-bit production signing key
│   │   └── src/main/AndroidManifest.xml # CAMERA, READ_MEDIA_IMAGES, INTERNET permissions
│   └── build.gradle                    # AndroidX dependency resolutionStrategy pin (1.13.1)
├── lib/
│   ├── main.dart                       # MultiProvider root, Material 3 Light/Dark theme configuration
│   ├── models/receipt.dart             # Receipt domain model: toMap(), fromMap(), copyWith()
│   ├── services/
│   │   ├── database_helper.dart        # SQLite helper (sqflite): receipts table CRUD & sync status
│   │   ├── receipt_regex_parser.dart   # Heuristic engine: Total, Date, Merchant, Category extraction
│   │   └── ocr_service.dart            # ML Kit TextRecognizer wrapper + local photo file storage
│   ├── providers/expense_provider.dart  # ChangeNotifier: CRUD state, sync queue, category & weekly data
│   ├── widgets/
│   │   ├── donut_chart.dart            # CustomPainter: Animated Donut Category Chart & Legend
│   │   ├── weekly_bar_chart.dart       # CustomPainter: 7-Day Animated Bar Chart with gridlines
│   │   └── review_receipt_dialog.dart  # M3 Dialog: Image preview, editable inputs & validation
│   └── screens/home_screen.dart        # Main dashboard: spending cards, chart switcher, receipt list, FAB
├── release/
│   ├── app-release.apk                 # Signed production APK (40.0 MB, apksigner verified)
│   └── VKU_MiniProject3_Report.pdf     # 4-page technical PDF report
├── test/
│   ├── receipt_regex_parser_test.dart  # Unit tests for Vietnamese & international receipts (100% pass)
│   └── widget_test.dart                # Smoke test for VKUExpenseApp widget tree
└── REPORT.md                           # Comprehensive technical report
```

State Management & OCR Pipeline Flow

```
[User triggers Scan (Camera/Gallery)]
        │
        ▼
[OcrService.saveReceiptPhoto()] ──────► Copied to /app_documents/receipt_photos/*.jpg
        │
        ▼
[ML Kit TextRecognizer.processImage()] ──► Raw recognized string (on-device)
        │
        ▼
[ReceiptRegexParser.parse()] ─────────────► Structured ParsedReceiptResult (Total, Date, Merchant, Category)
        │
        ▼
[ReviewReceiptDialog (UI)] ───────────────► User verifies, adjusts numbers or category, confirms
        │
        ▼
[ExpenseProvider.addReceipt()]
  ├── DatabaseHelper.insertReceipt() ─────► SQLite DB [sync_status: 'pending_sync']
  ├── _receipts.insert(0, receipt) ───────► Prepend to in-memory list
  └── notifyListeners() ──────────────────► Dashboard & CustomPainter charts animate updates
        │
        ▼
[User taps Sync on AppBar]
  ├── DatabaseHelper.markAllAsSynced() ──► Updates DB records to 'synced'
  └── notifyListeners() ──────────────────► Cloud status badge updates to 'Synced'
```

Exception Handling Strategies

- **Unclear / Poor Quality Receipt Photos:** `ReceiptRegexParser` employs a two-tier strategy: if no standard keyword line (`TỔNG CỘNG`, `TOTAL`) matches, it filters out noise and falls back to the largest plausible numerical value.
- **Tender vs Total Confusion:** Distinguishes between total expense and cash handed over by customer using explicit regex exclusion (`tiền mặt`, `tiền thối`, `cash`, `change`).

- **Camera Permission Denial:** `ImagePicker` wraps platform channel exceptions in try/catch; if permission is rejected, the app displays a descriptive SnackBar and allows fallback to Manual Entry.
- **Database IO Errors:** Database operations are wrapped with try/catch blocks; UI falls back gracefully without crashing, preserving current state.
- **R8 / ProGuard Stripping:** Added `-dontwarn com.google.mlkit.vision.text.**` to `proguard-rules.pro` so R8 does not fail on unused optional multi-language scripts.

4. EMPIRICAL EVIDENCE & SCREENSHOTS

The following screenshots illustrate key application states running on physical devices and mobile emulators:

- **Screenshot 1 — Main Dashboard & Animated Donut Chart:** Dashboard featuring Total Spending card, sync status badge, CustomPainter Donut chart with category breakdown and legend, and recent receipt transactions list.
- **Screenshot 2 — Weekly Trend Bar Chart (CustomPainter):** Custom canvas 7-day expense trend with rounded bar tops, dotted grid guidelines, dynamic max spend indicator, and highlight on the current day.
- **Screenshot 3 — Verification & Manual Correction Dialog:** Post-OCR review dialog showing captured receipt photo thumbnail, ML Kit extraction badge, editable Merchant name, numeric formatted Total Amount, DatePicker, and Category selector.
- **Screenshot 4 — Dark Mode & Sync Queue Integration:** Material 3 Dark Theme interface with orange 'pending sync' badges and quick theme switching via AppBar.

Test environment: Google Android 14 (API 34), Flutter 3.27.4, Dart 3.6.2, tested locally with Gradle 8.3 & Adoptium JDK 17.0.20. All unit and widget tests verified via `flutter test` (4/4 passed). Release APK verified using Android SDK `apksigner`.

5. TECHNICAL CHALLENGES & RESOLUTIONS

Challenge 1: Android Gradle Plugin (AGP 8.1.0) & AndroidX Dependency Version Mismatch

Problem: During the initial `flutter build apk --release` run, the build failed with:
`Dependency 'androidx.core:core-ktx:1.18.0' requires libraries and applications to compile against version 36 or later of the Android APIs. This build currently uses Android Gradle plugin 8.1.0.`
Recent transitive updates in Android libraries pulled in `androidx.core:core-ktx:1.18.0`, which enforces Android API 36, conflicting with Flutter 3.27's default compileSdk (34/35).

Resolution: Applied a targeted Gradle resolution strategy in `android/build.gradle` to force all transitive dependencies of `androidx.core:core` and `androidx.core:core-ktx` to remain on the stable `1.13.1` version:

```
allprojects {
    configurations.all {
        resolutionStrategy.eachDependency { DependencyResolveDetails details ->
            if (details.requested.group == 'androidx.core' &&
               (details.requested.name == 'core' || details.requested.name == 'core-ktx')) {
                details.useVersion '1.13.1'
            }
        }
    }
}
```

This cleanly bypassed the API 36 requirement without breaking any plugin functionality.

Challenge 2: ProGuard / R8 Missing Classes Failure with ML Kit Text Recognition

Problem: When building the obfuscated release APK (`:app:minifyReleaseWithR8`), R8 terminated with:
`ERROR: R8: Missing class com.google.mlkit.vision.text.chinese.ChineseTextRecognizerOptions`
Because `google_mlkit_text_recognition` supports multiple scripts (Latin, Chinese, Japanese, Korean, Devanagari) in its Java code, but only the Latin model artifact was included in `pubspec.yaml`, R8 flagged the missing optional script classes as build blockers.

Resolution: Created `android/app/proguard-rules.pro` containing explicit keep and dontwarn directives:

```
-dontwarn com.google.mlkit.vision.text.**
-dontwarn com.google.mlkit.**
-keep class com.google.mlkit.vision.text.** { *; }
-keep class com.google.mlkit.** { *; }
```

And linked it in `android/app/build.gradle` under `buildTypes.release`:

```
proguardFiles getDefaultProguardFile('proguard-android-optimize.txt'), 'proguard-rules.pro'
```

The R8 shrinker completed successfully, producing an optimized 40.0 MB release APK.

Challenge 3: Distinguishing Total Amount vs Cash Tendered in Vietnamese Receipts

Problem: In Vietnamese retail receipts (e.g., Highlands Coffee, Circle K), the text often contains both `TỔNG CỘNG: 68.000 đ` and `Tiền mặt: 100.000 đ` (followed by `Tiền thối: 32.000 đ`). Initial regex parsing matched the keyword `tiền mặt` as a total keyword, mistakenly extracting 100,000 VND instead of the actual 68,000 VND expense.

Resolution: Engineered a two-stage heuristic regex parser in `ReceiptRegexParser`:

1. Separated keywords into `_totalKeywordPattern` (`tổng cộng`, `tổng tiền`, `thành tiền`, `total`, `amount due`) and `_tenderKeywordPattern` (`tiền mặt`, `tiền thối`, `tiền thừa`, `cash`, `change`).
2. Strictly filtered out lines matching `_tenderKeywordPattern` during total extraction, and prioritized scanning the immediate line or next line for the currency figure.
3. Validated the fix with comprehensive unit tests in `test/receipt_regex_parser_test.dart`, confirming 100% accurate parsing across diverse receipt styles.

Report generated in compliance with VKU Cross-Platform Technical Standards • 2026